

Metro Passenger Display System

Real-Time GUI Display with Emergency Alert & Station Map

Technical Documentation

Prepared by:

Mohamad Omar Aboghalon

Language: Python 3 / Tkinter

Date: Dec 2025

Version: 1.0

1.Introduction

1.1 Project Overview

The Metro Passenger Display System is a real-time desktop application built with Python and the Tkinter GUI framework. It simulates the passenger information display screen found inside metro trains, showing the next upcoming station, the current direction of travel, station-specific information (in both Arabic and English), an animated station-map, and an emergency announcement banner.

The system targets the metro line running between Adly Mansour and the Administrative Capital in Cairo, Egypt — a corridor of eight stations. It is designed to run on any Windows, macOS, or Linux machine that has Python 3 installed, with zero external dependencies.

1.2 Objectives

- Display the next station name in both Arabic and English in real time.
- Animate a horizontal metro-line map with colour-coded visited / upcoming nodes.
- Support bidirectional travel (forward and backward) with a single button click.
- Trigger a blinking emergency announcement banner that can be toggled on and off.
- Show a live digital clock updated every second.
- Provide a separate operator control panel for station advancement, direction switching, and emergency alerts.
- Remain dependency-free — uses only Python standard library modules.

1.3 Tools & Technologies

Programming Language

- **Python 3.x:** Core language used for all logic, UI, and event handling.

GUI Framework

- **Tkinter:** Python's built-in GUI toolkit; used for all windows, labels, canvases, and buttons.

Standard-Library Modules

- **datetime:** Used to generate and update the live clock displayed in the corner of the screen.

IDE

- **Visual Studio Code / PyCharm:** Recommended environments for editing and running the script.

Other Tools

- **Python Interpreter:** Any standard CPython 3.6+ installation is sufficient to run the application.

2. System Architecture

2.1 High-Level Structure

The application is contained in a single Python script and is organised into the following logical sections:

Component	Description
Global State	Booleans that control blinking animation, emergency mode, and UI state.
Static Data	Hard-coded station list (Arabic + English names, info text) and direction labels.
Runtime State	Mutable values for <code>current_index</code> , <code>direction</code> , and <code>emergency_active</code> that change during operation.
Main Display Window	The full-screen Tkinter root window (display) containing all passenger-facing labels and canvas.
Control Panel	A secondary Toplevel window with three operator buttons.
Drawing Functions	<code>draw_map()</code> renders the station map on the canvas.
UI Update Logic	<code>update_display()</code> synchronises every label with the current runtime state.
Control Actions	Functions: <code>next_station()</code> , <code>switch_direction()</code> , <code>emergency_toggle()</code> .
Background Tasks	<code>update_time()</code> , <code>blink_next_station()</code> , <code>blink_alert_label()</code> — all scheduled via <code>display.after()</code> .

2.2 Window Layout

The passenger-facing display window is divided into vertical zones from top to bottom:

1. Metro-line canvas (horizontal station map with blinking next-station ring).
2. Emergency alert label (red, blinking when active).
3. Next-station label (large bold text, Arabic + English).
4. Direction label (shows origin → destination in both languages).
5. Information label (landmark / district info for the upcoming station).
6. Live clock (bottom-right corner, updated every second).



🚨 نداء طارئ | EMERGENCY ANNOUNCEMENT 🚨

المحطة التالية | Next Station:
المستقبل — Almustakbal

الاتجاه | Direction:
العاصمة → عدلي منصور | Adly Mansour → Capital

معلومات | Information:
قرية من جامعة المستقبل | Near Almustakbal University

2026-05-06 | 20:34:36

2.3 Global State Variables

Variable	Type / Value	Purpose
<code>blink_on</code>	bool	Toggles the blue ring around the next-station node on the canvas.
<code>alert_blink_on</code>	bool	Toggles the visibility of the emergency text label.
<code>alert_blinking</code>	bool	Prevents <code>blink_alert_label()</code> from rescheduling after emergency is cleared.
<code>current_index</code>	int (0-7)	Index into the stations list representing the upcoming station.
<code>direction</code>	str	Either 'forward' (Adly Mansour → Capital) or 'backward' (Capital → Adly Mansour).
<code>emergency_active</code>	bool	Master flag that enables/disables emergency mode.

3.Data Structures

3.1 Station List

Stations are stored as a Python list of dictionaries. Each dictionary contains four keys:

Component	Description
ar	Arabic display name of the station.
en	English display name of the station.
info_ar	Short Arabic description or nearby landmark.
info_en	Short English description or nearby landmark.

All eight stations in order:

English Name	Arabic Name	Info / Landmark
Adly Mansour	عدي منصور	Adly Mansour Interchange Station
Obour	العبور	Obour City and Carrefour Obour
Almustakbal	المستقبل	Near Almustakbal University
Alshorouk	الشروق	Alshorouk Park
Badr	بدر	Near Badr City
Asher of Ramadan	العاشر من رمضان	Busy residential district
Hadaik Alasima	حدائق العاصمة	Near the most famous compounds
Capital	العاصمة	Capital center and major facilities

3.2 Direction Dictionary

The DIRECTIONS dictionary provides the bilingual route labels shown in the direction zone of the display. It has two keys — 'forward' and 'backward' — each holding sub-keys 'ar' (Arabic) and 'en' (English).

```
DIRECTIONS = {
    "forward": { "ar": "عدي منصور → العاصمة", "en": "Adly Mansour → Capital" },
    "backward": { "ar": "العاصمة → عدي منصور", "en": "Capital → Adly Mansour" }
}
```

4.Functions & Logic

4.1 draw_map()

Redraws the entire canvas from scratch on every call. The canvas is first cleared with `canvas.delete('all')`, then each station is plotted at an evenly spaced x-position along the metro line:

- Stations already passed are coloured red; upcoming stations are coloured green.
- The node for `current_index` gets an additional animated blue ring when `blink_on` is `True`.
- Arabic names are rendered above the line; English names below.
- Connecting segments are drawn as grey lines between adjacent nodes.

The spacing calculation adapts to the window width, making the map fully responsive:

```
spacing = (width - 200) // (len(stations) - 1)
```

4.2 update_display()

Synchronises every visible label with the current runtime state in a single call:

- Sets `alert_label` text to the emergency message or an empty string.
- Updates `next_station_label` with the Arabic and English name of `stations[current_index]`.
- Updates `direction_label` with the bilingual route string from `DIRECTIONS[direction]`.
- Updates `info_label` with the bilingual info text of the current station.
- Calls `draw_map()` to refresh the canvas.
- Calls `display.update_idletasks()` to flush pending geometry updates.

4.3 next_station()

Advances or retreats the `current_index` by one position depending on the active direction, then calls `update_display()`. Boundary conditions prevent the index from going out of range:

```
if direction == 'forward' and current_index < len(stations) - 1:
    current_index += 1
elif direction == 'backward' and current_index > 0:
    current_index -= 1
```

4.4 switch_direction()

Flips the direction string between 'forward' and 'backward' and resets `current_index` to either 0 (start of forward run) or the last station index (start of backward run). `update_display()` is called afterwards.

4.5 emergency_toggle()

Toggles the emergency_active flag. When activated it sets alert_blinking = True and starts the blink_alert_label() loop. When deactivated it stops the loop and clears the alert label immediately.

4.6 Background Scheduling (after loops)

Three perpetual loops run concurrently via Tkinter's after() scheduler:

Component	Description
update_time()	Reads system time from datetime.now(), updates time_label, reschedules every 1 000 ms.
blink_next_station()	Flips blink_on and calls draw_map(), reschedules every 500 ms (always running).
blink_alert_label()	Flips alert_blink_on and updates alert_label text, reschedules every 500 ms only while alert_blinking is True.



5.Application User Guide

5.1 Running the Application

7. Ensure Python 3.x is installed on your machine.
8. Open a terminal (or Command Prompt on Windows).
9. Navigate to the project folder and run: `python metro_display.py`
10. Two windows open: the full-screen passenger display and the smaller control panel.

5.2 Control Panel Buttons

Component	Description
Next Station	Advances the display to the next station in the current direction of travel.
Switch Direction	Reverses the direction of travel and jumps to the first station of the new route.
Emergency (red)	Toggles the red blinking emergency announcement banner on/off.

5.3 Passenger Display Zones

The passenger-facing window requires no interaction. It automatically updates whenever an operator presses a control panel button. The display includes:

- A colour-coded metro-line map (red = passed, green = upcoming, blue ring = next stop).
- The name of the next station in large text, bilingual (Arabic and English).
- The current direction of travel, bilingual.
- Useful landmark or district information for the next station.
- A live clock in the bottom-right corner.
- A flashing red emergency banner when emergency mode is active.

6.Code Overview

6.1 Imports & Dependencies

```
import tkinter as tk
from datetime import datetime
```

No third-party packages are required. Both modules are part of the Python standard library.

6.2 Window Configuration

```
display = tk.Tk()
display.title("Metro Passenger Display")
display.configure(bg="#F5F5F2")
display.state("zoomed")
display.minsize(900, 500)
```

The main window is maximised on startup (zoomed). A minimum size prevents the station map from becoming unreadable on very small screens.

6.3 Canvas Responsiveness

The canvas is bound to the window's <Configure> event so that draw_map() is triggered automatically whenever the passenger resizes or moves the window:

```
display.bind("<Configure>", lambda e: draw_map())
```

6.4 Application Entry Point

```
update_display()
display.bind("<Configure>", lambda e: draw_map())
update_time()
blink_next_station()
display.mainloop()
```

update_display() and the background loops are started before mainloop() so the screen is never blank when the application first opens.



Imaginary image

Summary & Conclusion

The Metro Passenger Display System demonstrates how a fully functional, bilingual, real-time information display can be built using Python and Tkinter alone — without any external libraries or complex infrastructure. The application covers several important programming concepts:

- Event-driven GUI programming with a main event loop and callback-based user interaction.
- Perpetual background tasks scheduled through Tkinter's `after()` mechanism rather than raw threads, avoiding concurrency issues.
- Dynamic canvas drawing that adapts to window dimensions, producing a responsive layout.
- Bilingual content management, storing both Arabic and English text in structured data and rendering them simultaneously.
- State machine logic for direction switching and emergency toggling.

This project serves as both a practical prototype of a metro passenger information system and a clear educational example of real-world Python GUI development. It can be extended with additional features such as audio announcements, GPS-based automatic station detection, network-connected live data feeds, or integration with a central control room server.

— End of Documentation —